


# Ethernet Switching

*Marian Flavius-Andrei*
*AN 4 TI*
*AC UPT*

# Cuprins

1. Ethernet și frame-ul Ethernet
2. Adresa MAC
3. Tabela MAC
4. Metode de forwarding și setări de port

# 1.Ethernet și frame-ul Ethernet

- Ethernet e una din cele două tehnologii LAN folosite azi (cealaltă e WLAN). Funcționează pe cupru, fibră și coaxial.

- Lucrează la **Layer 1 și Layer 2** și e definit de **IEEE 802.2** (LLC) și **802.3** (MAC).

- Viteze: 10 Mbps, 100 Mbps, 1, 10, 40 și 100 Gbps.

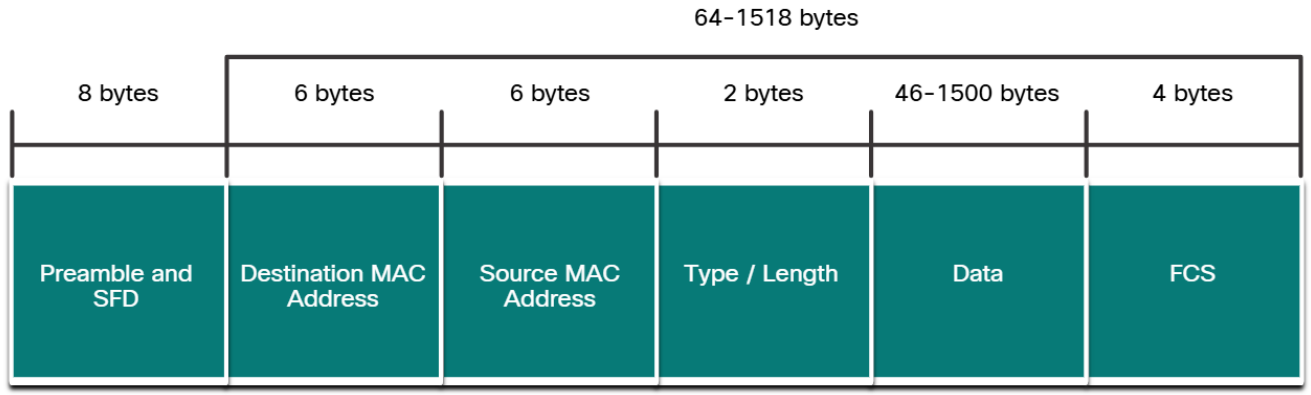
- Substratul **MAC** face încapsularea: structura frame-ului, adresarea MAC-la-MAC în același LAN și detecția erorilor prin **FCS**.

- Variante 802.3: **802.3u** Fast Ethernet, **802.3z** Gigabit pe fibră, **802.3ab** Gigabit pe cupru, **802.3ae** 10 Gigabit pe fibră.

- **CSMA/CD** există doar la Ethernet-ul legacy (hub-uri, half-duplex) și include un algoritm de **back-off** pentru retransmisie. Switch-urile moderne sunt full-duplex și **nu au nevoie de CSMA/CD**.

## Structura frame-ului Ethernet

| Câmp | Mărime | Rol |
|---|---|---|
| Preamble + SFD | 7 + 1 bytes | Sincronizare, anunță receptorul că vine un frame |
| MAC destinație | 6 bytes | Cui îi e adresat (unicast, multicast sau broadcast) |
| MAC sursă | 6 bytes | NIC-ul care a trimis |
| Type / Length (EtherType) | 2 bytes | Protocolul L3 din interior: 0x0800 = IPv4, 0x86DD = IPv6, 0x0806 = ARP |
| Data | 46–1500 bytes | Pachetul L3. Dacă e prea mic, se completează cu padding |
| FCS | 4 bytes | CRC pentru detecția erorilor |



- Dimensiunea frame-ului: **minim 64 bytes, maxim 1518 bytes**, numărate de la MAC destinație până la FCS. **Preamble-ul nu se pune la socoteală.**

- Sub 64 bytes → **runt** sau „collision fragment”, aruncat automat.

- Peste 1500 bytes de date → **jumbo** sau „baby giant”. Majoritatea switch-urilor și NIC-urilor Fast și Gigabit Ethernet le suportă.

- Frame-urile prea mici sau prea mari sunt aruncate și sunt de obicei urmarea unor coliziuni sau semnale nedorite.

- Frame-ul poate fi mai mare dacă se adaugă **tag de VLAN**.

# 2. Adresa MAC

- **48 de biți = 6 bytes = 12 cifre hexa** (o cifră hexa = 4 biți). Hexa e folosit și la IPv6.

- Notații hexa: zerourile din față se scriu mereu (0A, nu A), iar valorile apar ca 0x73 sau 73H. La conversii treci întotdeauna prin binar.

- Structura:
  - primii 6 hexa = **OUI**, codul producătorului, alocat de **IEEE**
  - ultimii 6 hexa = valoare unică dată de producător
  - Exemplu: OUI Cisco 00-60-2F + 3A-07-BC = 00-60-2F-3A-07-BC

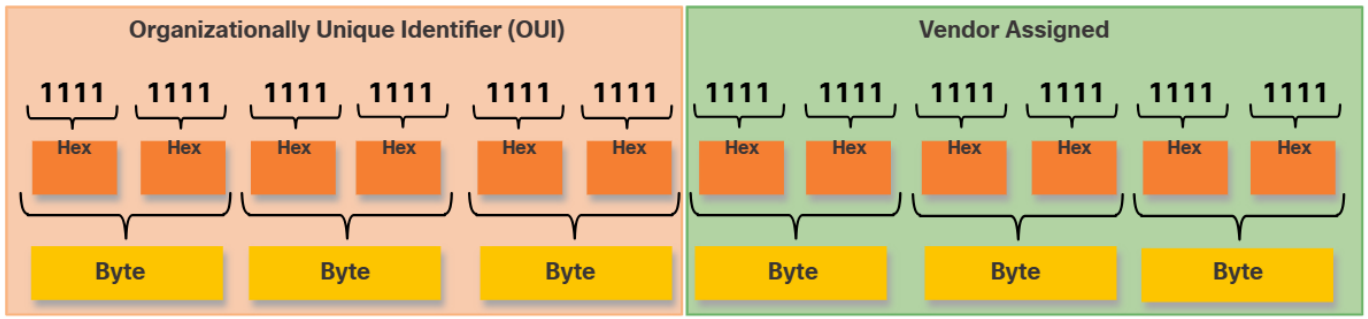
- Producătorul răspunde de unicitate, dar pot apărea MAC-uri duplicate din erori de fabricație, la mașini virtuale sau prin modificări software.

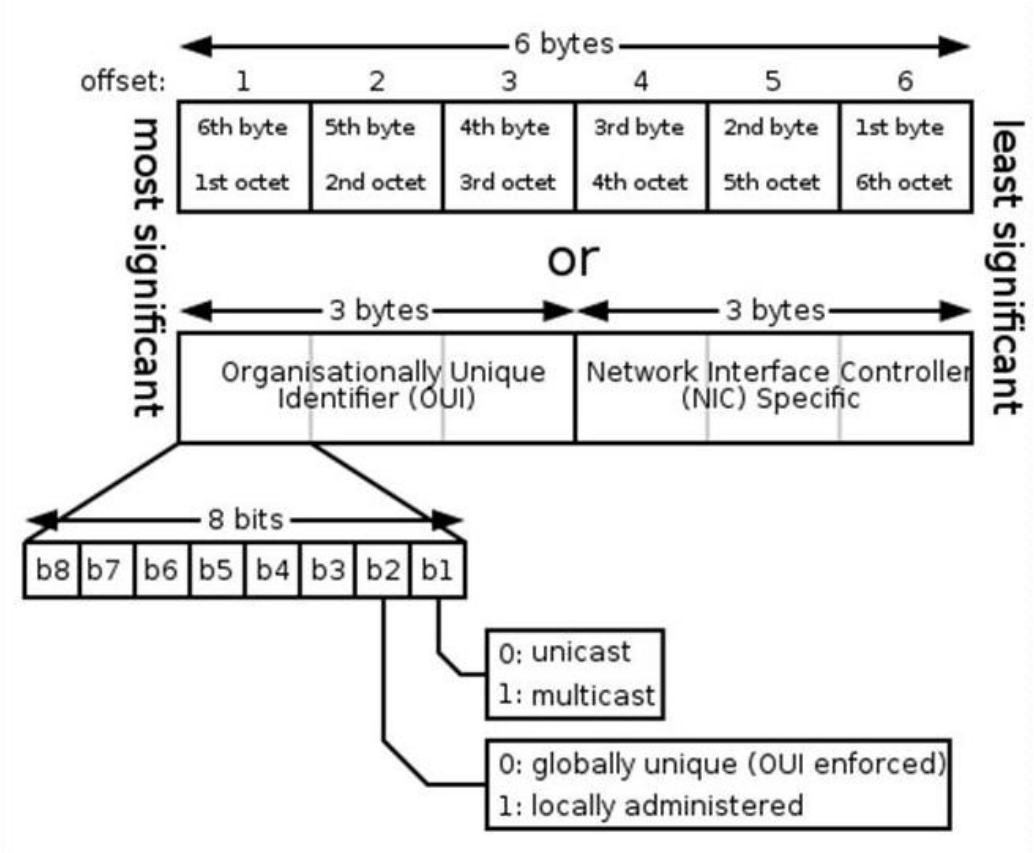
- Se mai numește **BIA** (Burned-In Address), pentru că e scrisă în ROM-ul NIC-ului. La pornire, NIC-ul o copiază în RAM.

- Capcană: MAC-ul **se poate schimba din software**, deci **filtrarea pe bază de MAC nu e o metodă sigură**.

- Când NIC-ul primește un frame, compară MAC-ul destinație cu al lui. Dacă nu se potrivește, îl aruncă. Dacă se potrivește, îl urcă în stivă. Acceptă și frame-urile **broadcast** și pe cele **multicast** din grupurile din care face parte.

- Orice dispozitiv care trimite sau primește frame-uri Ethernet are un MAC: PC-uri, servere, imprimante, routere.


Organizationally Unique Identifier (OUI)
Vendor Assigned
1111
Hex
Byte


6 bytes
offset:
1
2
3
4
5
6
6th byte
1st octet
5th byte
2nd octet
4th byte
3rd octet
3rd byte
4th octet
2nd byte
5th octet
1st byte
6th octet
most significant
least significant
or
3 bytes
3 bytes
Organisationally Unique Identifier (OUI)
Network Interface Controller (NIC) Specific
8 bits
b8 b7 b6 b5 b4 b3 b2 b1
0: unicast
1: multicast
0: globally unique (OUI enforced)
1: locally administered

# Cele 3 tipuri de MAC destinație:

| Tip | MAC destinație | Cum se comportă |
|---|---|---|
| Unicast | MAC-ul unui singur dispozitiv | Livrat unui singur host. Pentru IPv4 se află prin ARP, pentru IPv6 prin ND |
| Broadcast | FF-FF-FF-FF-FF-FF (48 de 1) | Trimis pe toate porturile, mai puțin cel de intrare. Nu trece de router. Exemple: DHCPv4, ARP Request |
| Multicast | 01-00-5E-... pentru IPv4 multicast, 33-33-... pentru IPv6 multicast | Ajunge la un grup. Switch-ul îl trimite pe toate porturile (în afară de cazul în care are multicast snooping). Nu trece de router decât dacă routerul e configurat să ruteze multicast |

- Capcană: **MAC-ul sursă e întotdeauna unicast**.
- IP broadcast = toți biții de host pe 1, de exemplu 192.168.1.255.
- **ARP Request** e trimis ca broadcast Ethernet, deși nu conține un pachet IPv4.
- Multicast IPv4 = **224.0.0.0 – 239.255.255.255**. Multicast IPv6 începe cu **ff00::/8**.
- Există și MAC-uri multicast rezervate pentru protocoale care nu sunt IP, cum sunt **STP** și **LLDP**.
- Multicast-ul e folosit de protocoalele de rutare și de unele aplicații video.

# 3. Tabela MAC

- Switch-ul ia decizii **doar pe baza MAC-urilor** de la Layer 2 și nu se uită la ce conține frame-ul.

- Tabela MAC se mai numește **CAM table**. La pornire e goală.

**Cum funcționează:**

1. **Learn**: se uită la **MAC-ul sursă** și la portul pe care a intrat frame-ul.
    1. MAC nou → îl adaugă în tabelă, cu portul respectiv.
    2. MAC existent → resetează timer-ul intrării (implicit **5 minute**).
    3. Același MAC pe alt port → înlocuiește intrarea cu portul nou.

2. **Forward**: se uită la **MAC-ul destinație**.
    1. Destinația e în tabelă → trimite **doar pe portul respectiv** (asta se numește **filtrare**).
    2. Destinația nu e în tabelă → **unknown unicast**, trimis pe **toate porturile, mai puțin cel de intrare** (flooding).
    3. Broadcast sau multicast → tot flooding, pe toate porturile mai puțin cel de intrare.

Pe scurt: **învață din sursă, decide după destinație**.

# 4. Metode de forwarding și setări de port

| Metodă | Cum funcționează | Avantaj / dezavantaj |
|---|---|---|
| Store-and-forward | Primește tot frame-ul, verifică CRC-ul, apoi îl trimite | Aruncă frame-urile cu erori, deci nu irosește bandwidth. Necesar pentru QoS (de exemplu prioritate pentru VoIP) |
| Cut-through | Trimite frame-ul înainte să-l primească complet, imediat ce citește MAC-ul destinație | Latență mică, dar fără verificare de erori |
| └ Fast-forward | Trimite imediat după MAC-ul destinație | Cea mai mică latență. E varianta tipică de cut-through. NIC-ul destinației aruncă frame-urile corupte |
| └ Fragment-free | Așteaptă primii 64 bytes | Compromis: majoritatea erorilor și coliziunilor apar în primii 64 bytes |

- Unele switch-uri fac cut-through până trece un prag de erori, apoi trec pe store-and-forward, și invers când erorile scad.

**Buffering:**

- **Port-based**: cozi legate de porturi. Un singur frame blocat la un port ocupat le întârzie pe toate din spatele lui.

- **Shared memory**: un buffer comun, alocat dinamic. Pierde mai puține frame-uri și e important pentru **switching asimetric**, adică porturi cu viteze diferite (server pe 10 Gbps, PC-uri pe 1 Gbps).

**Duplex și viteză:**

- Cele două capete trebuie să aibă **aceleași setări**.

- **Autonegocierea** alege cea mai bună combinație comună: full-duplex dacă ambele capete pot, plus cea mai mare viteză comună. E activată implicit pe majoritatea switch-urilor Cisco și NIC-urilor.

- **Porturile Gigabit funcționează doar full-duplex.**

- **Duplex mismatch** e una dintre cele mai frecvente cauze de probleme de performanță pe legăturile 10/100. Capătul half-duplex vede coliziuni continue, pentru că celălalt trimite oricând. Apare după un reset al porturilor sau când reconfigurezi doar un capăt.

- Regulă: autonegociere pe ambele capete sau pe niciunul. Best practice: ambele porturi de switch în full-duplex.

**Auto-MDIX:**

- Detectează automat tipul de cablu și se adaptează, deci merge și straight-through, și crossover.
- E activat implicit din IOS 12.2(18)SE. Se reactivează cu mdix auto.
- Cursul recomandă totuși cablul corect. Notă: **router–host direct cere crossover**.